

PollPulse - Student Voice, Feedback and Election Portal

Cover Page

Project Title: PollPulse - Student Voice, Feedback and Election Portal

Course: Web Technologies

Submission Deadline: June 01, 2026

Frontend: Angular + TypeScript + HTML + CSS

Backend: ASP.NET Core Web API using C#

Database: MongoDB

Team Details: Replace with final team member names, roll numbers and GitHub usernames before final submission.

1. Introduction and Problem Statement

Universities need a structured way to collect student feedback and conduct class representative elections. Manual feedback and voting processes are difficult to track, slow to summarize and weak in transparency. PollPulse solves this by providing a single student portal for authentication, elections, voting, feedback and sentiment analysis, with a separate admin portal for reporting and management.

2. Objectives

- Provide student registration and secure login.
- Show class-specific dashboard data.
- Conduct CR/GR elections for specific degree/year/section groups.
- Enforce one vote per student per election.
- Collect category-wise student feedback.
- Analyze feedback sentiment as Positive, Neutral or Negative.
- Provide admin dashboards, reports and audit logs.
- Store all records in a real MongoDB database with seed data.

3. System Architecture

The project follows a three-layer architecture:

1. Angular frontend: pages, routing, guards, services, reactive forms and responsive UI.
2. ASP.NET Core backend: REST API controllers, JWT authentication, services and validation.
3. MongoDB database: collections for students, admins, elections, candidates, votes, feedback, classes, teachers, courses, events and audit logs.

High-level flow:

Browser -> Angular Components -> Angular Services -> ASP.NET Core Controllers -> MongoDBService -> MongoDB Collections

4. Frontend Implementation

The frontend is built in Angular with standalone components. It uses Angular Router for navigation and dynamic routes such as `/elections/:id`. Auth guards protect student and admin routes. Reactive Forms are used for login, signup, admin login, contact, feedback, voting and admin management forms. Angular HttpClient services connect the UI to backend APIs.

Main pages:

- Home / Landing Page
- Login and Signup
- Student Dashboard
- Elections List
- Election Detail
- Election Results
- Feedback Pages
- Admin Login
- Admin Dashboard
- Admin Manage Elections
- Admin Masters
- Admin Feedback
- Admin Sentiment
- Admin Audit Logs

5. Backend Implementation

The backend is built with ASP.NET Core Web API and C#. JWT Bearer authentication protects student and admin routes. The backend validates requests and returns proper HTTP status codes such as 200, 201, 400, 401, 403, 404 and 409.

Main controllers:

- AuthController
- ElectionController
- FeedbackController
- AdminController
- PublicController
- AdminTeachersController
- AdminCoursesController
- AdminEventsController
- AdminClassSectionsController

Full CRUD is demonstrated by four dedicated controllers:

- /api/admin/crud/teachers
- /api/admin/crud/courses
- /api/admin/crud/events
- /api/admin/crud/classes

6. Database Design

MongoDB database name: au_cs_voice_portal

Main collections:

- admins
- users
- classections
- teachers
- courses
- events
- elections
- candidates
- votes
- feedbacks
- auditlogs

Relationships are maintained using ObjectId references. For example, candidates reference elections, votes reference users/elections/candidates, and feedback references users plus optional teacher/course/event targets. A unique index on election + user prevents duplicate voting.

7. API Documentation Summary

Authentication endpoints:

- POST /api/auth/register
- POST /api/auth/login

- POST /api/auth/admin/login

Student endpoints:

- GET /api/elections/dashboard
- GET /api/elections
- GET /api/elections/{id}
- POST /api/elections/{id}/vote
- GET /api/elections/{id}/results
- GET /api/feedbacks/masters
- GET /api/feedbacks/recent
- POST /api/feedbacks

Admin endpoints:

- GET /api/admin/dashboard
- GET /api/admin/elections
- POST /api/admin/elections
- DELETE /api/admin/elections/{id}
- GET /api/admin/feedback
- GET /api/admin/sentiment
- GET /api/admin/audit-logs
- Full CRUD endpoints under /api/admin/crud/...

Detailed API documentation is included in report/API_DOCUMENTATION.md.

8. UI Screenshots

Screenshots must be captured after running the app locally. The screenshot checklist is available in report/screenshots/README.md. Add screenshots for all major student and admin pages before final GitHub/LMS submission.

9. Deployment Steps

Local deployment steps:

1. Install Node.js LTS, Angular CLI, .NET 8 SDK and MongoDB.
2. Start MongoDB.
3. Run backend:

```
cd backend
dotnet restore
dotnet run --launch-profile https
```

4. Run frontend:

```
cd frontend
npm install
npm start
```

5. Open `http://localhost:4200`.

Optional production deployment:

- Deploy Angular frontend on Netlify or Vercel.
- Deploy ASP.NET Core backend on Azure, Railway or Render.
- Use MongoDB Atlas for cloud database.
- Update frontend API base URLs for deployed backend.

10. Testing and Validation

Test cases:

- Student registration validates university email format.
- Student login returns JWT and opens dashboard.
- Student only sees elections for own class section.
- Student can vote once per election only.
- Feedback form validates required fields.
- Feedback submission creates sentiment result.
- Admin login opens protected admin dashboard.
- Admin can create and delete elections.
- Admin can create teacher/course/event master records.
- Admin can view feedback, sentiment and audit logs.

11. Requirement Mapping

Requirement	Implementation
Angular frontend	frontend/
ASP.NET Core backend	backend/
MongoDB database	MongoDBService.cs and database/
5-6 dynamic pages	More than 10 dynamic pages
Login/signup validation	Reactive Forms and JWT
Dashboard dynamic data	Student and admin dashboards
Main feature page	Elections and feedback modules
Detail route	/elections/:id
Admin CRUD	Admin elections and master data
4 CRUD controllers	Teachers, courses, events, classes
5+ collections	10+ MongoDB collections
Seed data	Automatic seed on backend startup
Report	report/Pollpulse_Project_Report.pdf
Video demo	Script and link placeholder in demo/

12. Conclusion

PollPulse fulfills the semester project objective by implementing a complete, database-driven, full-stack web application using the required technologies. It includes authentication, dynamic Angular pages, REST APIs, MongoDB collections, seed data, CRUD controllers, report documentation and a demo guide.